

AI-MARKETER PLATFORM OS V4.0

Unified Hourly Analytics Calculation Reference

Complete formula documentation for all metrics displayed
in the Cross-Platform Analytics dashboard

Data source: **ad_metrics** table (PostgreSQL) | Prepared: April 2026 | Confidential — Internal Use Only

TABLE OF CONTENTS

1. Global Data Anchor — How the time window is determined
2. KPI Bar Cards — Impressions, Views, Engagement Rate, CTR, Followers, Revenue + Delta Arrows
3. Hourly Trends Chart — Time-series line chart calculations
4. Post Performance — Last Hour — Per-ad Views, ER, Velocity, Virality, Sparkline
5. Virality Detection — Lift score, Spike alerts, Engagement velocity
6. Audience Activity Heatmap — Hourly intensity grid
7. Video Deep Insights — Avg Watch, Completion Rate, Retention Curve

Global Data Anchor — Applies to ALL Sections

Before any calculation runs, the system finds the **most recent timestamp that contains real ad delivery**. This anchor is used instead of the server's live clock to prevent Facebook's zero-delivery sync rows (inserted for the in-progress current hour) from pushing every window past all actual data.

```
maxDate = MAX( date ) FROM ad_metrics
WHERE ( impressions > 0 OR clicks > 0 OR spend > 0 )
AND company_id = <current company>
AND platform = <selected platform, or all>
```

```
If no rows match → maxDate = DateTime.UtcNow (safe fallback)
```

All "Current 1H" windows are [maxDate - 1 hour, maxDate]. All "Previous 1H" windows are [maxDate - 2 hours, maxDate - 1 hour].

1 KPI Bar Cards

Six summary cards appear at the top of the analytics page. Each shows a **current-hour value** and a **delta arrow** comparing it to the previous hour. All sums are drawn from rows where $\text{date} \in [\text{maxDate} - 1\text{h}, \text{maxDate}]$.

Bucket	Time Range
Current (1H)	$\text{maxDate} - 1 \text{ hour} \rightarrow \text{maxDate}$
Previous (1H)	$\text{maxDate} - 2 \text{ hours} \rightarrow \text{maxDate} - 1 \text{ hour}$

A. Impressions (1H)

Total number of times ads were displayed in the current hour.

```
Impressions = SUM( ad_metrics.impressions )
              WHERE date ∈ Current (1H) window
```

B. Views (1H)

Uses *video views* when available (YouTube, TikTok, Facebook video ads); falls back to *impressions* for image/static ads.

```
videoViews = SUM( ad_metrics.video_views )   WHERE date ∈ Current (1H) window

Views = IF videoViews > 0
        THEN videoViews
        ELSE impressions
```

C. Engagement Rate

Engagement is the sum of all social interactions. Rate is normalised against impressions so it is comparable across different ad sizes.

```
engagement = SUM(likes) + SUM(comments) + SUM(shares) + SUM(saves)
              WHERE date ∈ Current (1H) window

ER = engagement ÷ impressions × 100%
    = 0% if impressions = 0
```

Example: likes=120, comments=30, shares=18, saves=12 → engagement=180

impressions=4,800 → ER = $180 \div 4,800 \times 100 = 3.75\%$

D. CTR — Click-Through Rate

Measures how often people click an ad after seeing it.

```
CTR = SUM( clicks ) ÷ SUM( impressions ) × 100%
      = 0% if impressions = 0
```

Example: clicks=96, impressions=4,800

CTR = $96 \div 4,800 \times 100 = 2.00\%$

E. Followers Gained

Total new followers attributed to the ads in the current hour.

```
Followers = SUM( ad_metrics.followers_gained )
            WHERE date ∈ Current (1H) window
```

F. Revenue (1H)

Total conversion value generated in the current hour. For Facebook: derived from conversion actions multiplied by an average order value.

```
Revenue = SUM( ad_metrics.conversion_value )
          WHERE date ∈ Current (1H) window
```

G. Delta Arrow (▲▼ vs prev hr)

Shown in small text below each card value. Compares the current hour to the previous hour.

```
delta = ( current_value - previous_value ) ÷ previous_value × 100%
```

Display:

```
▲ X.X% vs prev hr  — green  — if delta ≥ 0
▼ X.X% vs prev hr  — red    — if delta < 0
—                  — grey   — if previous_value = 0
```

Example: current impressions = 5,200 · previous = 4,000

delta = $(5,200 - 4,000) \div 4,000 \times 100 = \text{▲ } 30.0\% \text{ vs prev hr}$

2

Hourly Trends Chart

The line chart shows metric volume over time. The user can toggle between *Views*, *Engagement*, *Watch time*, and *Followers*, and switch between *Combined* (all platforms, one line) and *By Platform* (one line per platform).

A. Time Window

```
window = last <N> hours from DateTime.UtcNow
default N = 48 hours    (selectable: 24h or 48h via the Range toggle)
```

B. Hour Bucketing & Timezone

Each raw database row has a UTC timestamp. Before grouping, **6 hours are added** (UTC → BST / GMT+6) so the X-axis labels match local business time.

```
hourKey = ( record.Date + 6 hours ) truncated to the start of that hour

Example: record.Date = 2026-04-27 07:30 UTC
        hourKey      = 2026-04-27 13:00 BST    (displayed as "Apr 27, 01 PM")
```

C. Metric Value per Hour Bucket

The selected tab determines which field is summed within each hour bucket:

Tab	Per-row value	Aggregation
Views (default)	videoViews > 0 ? videoViews : impressions	SUM per hour
Engagement	likes + comments + shares + saves	SUM per hour
Watch time	video_views × avg_watch_seconds	SUM per hour (total seconds)
Followers	followers_gained	SUM per hour

Why video_views × avg_watch_seconds?
Facebook reports *average* watch time per view. Multiplying by views gives total watch-seconds for that hour bucket, making platform comparison fair regardless of audience size.

D. Combined vs By Platform

```
Combined → group by hourKey only
          → one data series: value = SUM across all platforms
```

```
By Platform → group by (platform, hourKey)  
→ one data series per platform  
→ missing platform+hour combinations = 0
```

3 Post Performance — Last Hour

Shows per-ad metrics for the last hour of data. Only rows where `ad_id` IS NOT NULL are included — this means only ads that were synced from the ad platform (Facebook) with a proper Ad ID. Simulated rows without an `ad_id` are excluded.

A. Per-Ad Time Windows

```
anchor = MAX( date ) FROM ad_metrics
WHERE ad_id IS NOT NULL
      AND ( impressions > 0 OR clicks > 0 )
      AND company_id = <current company>

last window = [ anchor - 1h, anchor ]      ← "this hour"
prev window = [ anchor - 2h, anchor - 1h ] ← "previous hour"

Data fetched: all rows for that ad in the last 48-hour window
```

B. Views (1H)

```
lastViews = SUM( videoViews > 0 ? videoViews : impressions )
           for all rows of THIS AD in the `last` window
```

Example: Ad "BU AU LECTURE SERIES 4 LAW" has 3 rows in the last window:

row1: videoViews=100, row2: videoViews=95, row3: videoViews=96

lastViews = 100 + 95 + 96 = **291**

C. ER — Engagement Rate

```
lastImpr = SUM( impressions )           in `last` window for this ad
lastEng  = SUM( likes + comments + shares + saves ) in `last` window for this ad

ER = lastEng ÷ lastImpr × 100%
    = 0% if lastImpr = 0
```

Example: lastImpr=42,300 · lastEng=291 (likes=210, comments=26, shares=0, saves=55)

ER = 291 ÷ 42,300 × 100 = **0.69%**

D. Velocity & Velocity %

Velocity is simply the last-hour view count expressed as a per-hour rate. **Velocity %** shows how much that rate changed compared to the previous hour.

```
velocity      = lastViews      (same number as Views 1H)

prevViews     = SUM( videoViews > 0 ? videoViews : impressions )
               in `prev` window for this ad

velocityPct   = ( lastViews - prevViews ) ÷ prevViews × 100%
               = 0% if prevViews = 0

Display: "291/hr (+23%)"
  → 291 = velocity
  → +23% = velocityPct
```

Example: lastViews=291 · prevViews=236
 velocityPct = (291 - 236) ÷ 236 × 100 = **+23.3%** → shown as "(23%)"

E. Virality Score

Measures the share of viewers who shared the ad — the most reliable proxy for organic amplification.

```
lastShares = SUM( shares )      in `last` window for this ad
lastImpr   = SUM( impressions ) in `last` window for this ad

virality    = lastShares ÷ lastImpr × 100%
             = 0% if lastImpr = 0
```

F. Trending 🔥 / Dropping ⚠️ Flags

```
🔥 trending = ( velocityPct > +50% ) → views grew more than 50% vs prev hour
⚠️ dropping = ( velocityPct < -50% ) → views fell more than 50% vs prev hour
(no flag)  otherwise
```

G. Sparkline (mini chart)

```
Take the last 12 rows for this ad (ordered oldest → newest)
Each point = videoViews > 0 ? videoViews : impressions

Normalised to a 80×24 pixel SVG path for display.
```


H. Ad Age

```
ageHours = FLOOR( ( DateTime.UtcNow - ad.created_at ).TotalHours )
```

Example: ad created 2026-04-21 08:00 UTC · now = 2026-04-27 02:00 UTC

ageHours = floor(138.0) = **137h old**

4 Virality Detection

Identifies ads experiencing a sudden delivery surge by comparing their recent hourly rate to their baseline hourly rate over the prior 6 hours. Returns the top 5 risers and flags any spike above +200%.

A. Time Windows

```
anchor          = MAX( date ) WHERE impressions > 0 OR clicks > 0   (same global anchor)

recentStart     = anchor - 2h
baselineStart   = anchor - 8h

Recent period   = [ recentStart,   anchor ]           → last 2 hours
Baseline period = [ baselineStart, recentStart )      → 6 hours before recent
```

B. Recent Views (last 2 hours)

```
recent = SUM( videoViews > 0 ? videoViews : impressions )
        WHERE date ≥ recentStart
        AND ad_id = <this ad>
```

C. Baseline Rate (per hour, over the prior 6 hours)

```
baselineTotal = SUM( videoViews > 0 ? videoViews : impressions )
                WHERE date ≥ baselineStart
                AND date < recentStart
                AND ad_id = <this ad>

baseline = baselineTotal ÷ 6.0    ← average views per hour in the baseline period
```

D. Lift Score (Virality Index)

Compares the *recent hourly rate* to the *baseline hourly rate*. A lift of 1.0 means 100% above baseline (double the delivery).

```
recentRate = recent ÷ 2.0          ← average views per hour during recent period

lift = ( recentRate - baseline ) ÷ baseline
      = 0    if baseline = 0
```

Lift value	Meaning
------------	---------

0.0	Same delivery rate as baseline — no change
0.5	50% above baseline (1.5× the baseline rate)
1.0	100% above baseline (2× the baseline rate)
> 2.0	Spike alert triggered (3× the baseline rate)
-0.5	50% below baseline — delivery slowing down

Example: baseline = 500 views/hr · recent = 1,800 views over 2 hours

recentRate = $1,800 \div 2 = 900/\text{hr}$

lift = $(900 - 500) \div 500 = \mathbf{0.80 (+80\% \text{ above baseline})}$

E. Engagement Velocity

```
engagementVel = SUM( likes + comments + shares + saves )
                WHERE date ≥ recentStart
                AND ad_id = <this ad>
```

F. Spike Alert Threshold

```
Spike alert = TRUE if lift > 2.0
```

Meaning: the ad's recent hourly rate is MORE than 3× its 6-hour baseline.

"No spikes above +200%" is shown when no ad exceeds this threshold.

G. Top Risers Ranking

```
Sort all ads by lift (descending)
```

```
Return TOP 5
```

```
Displayed as: ad name, platform, lift%, engagementVelocity
```

5 Audience Activity Heatmap

A 24-column × N-platform grid showing at which hours each platform had the highest delivery intensity. Darker = more activity.

A. Time Window

```
since = DateTime.UtcNow - 48 hours (fixed; no anchor needed here)
```

B. Hour-of-Day Extraction (BST)

```
hour = ( record.Date + 6 hours ).Hour ← integer 0-23 in BST (UTC+6)
```

This makes column "11" on the heatmap = 11:00–11:59 BST local time.

C. Cell Value (Intensity)

For each (platform, hour) group:

```
cellValue = AVERAGE(
    impressions > 0 ? impressions : engagement
)
```

where engagement = likes + comments + shares + saves

Impressions are preferred because engagement is often 0 for paid ads even when they are actively delivering — impressions give a more meaningful heat signal.

Example — Google Ads, hour 11 (BST):

Rows in that bucket: [1,200], [980], [1,450], [1,100]

cellValue = $(1,200 + 980 + 1,450 + 1,100) \div 4 = 1,182.5$

This is normalised visually — the darkest cell on each row = highest average in that platform.

D. Output Structure

Result: array of platforms, each with 24 values (one per hour 0–23)

Missing hours (no data) → value = 0 → displayed as lightest shade

6

Video Deep Insights

Shown for the selected ad in the dropdown. Reads the **most recent** `ad_metrics` row for that ad (ordered by date descending).

A. Avg Watch (seconds)

```
avgWatchSeconds = ad_metrics.avg_watch_seconds
                  (most recent row for this ad)

For Facebook: sourced from video_avg_time_watched_actions in the Graph API.
Displayed as seconds (e.g. "0s") or "Xm Ys" for values ≥ 60s.
```

B. Completion Rate

```
views      = ad_metrics.video_views      (most recent row)
completions = ad_metrics.video_completions (most recent row)

completion = completions ÷ views × 100%
            = 0% if views = 0
```

Example: views=5,000 · completions=1,200
completion = 1,200 ÷ 5,000 × 100 = **24.00%**

C. Retention Curve (0% → 100% of video)

The retention curve is modelled as a **decile-decayed exponential**. It plots viewer retention at 11 equally spaced points (every 10% of video length). The curve shape approximates typical video ad drop-off behaviour.

```
For x = 0.0, 0.1, 0.2, ... 1.0 (10 deciles):

retention(x) = MAX( 5,  e^(-x × 1.6) × 100 )

Floor of 5% ensures the curve never shows complete abandonment.
```

Video position	x value	Formula	Retention %
0% (start)	0.0	$e^{(0)} \times 100$	100.00%
10%	0.1	$e^{(-0.16)} \times 100$	85.21%

20%	0.2	$e^{(-0.32)} \times 100$	72.61%
30%	0.3	$e^{(-0.48)} \times 100$	61.88%
40%	0.4	$e^{(-0.64)} \times 100$	52.73%
50%	0.5	$e^{(-0.80)} \times 100$	44.93%
60%	0.6	$e^{(-0.96)} \times 100$	38.29%
70%	0.7	$e^{(-1.12)} \times 100$	32.63%
80%	0.8	$e^{(-1.28)} \times 100$	27.80%
90%	0.9	$e^{(-1.44)} \times 100$	23.69%
100% (end)	1.0	$e^{(-1.60)} \times 100$	20.19%

Note on the 50% Retention figure (44.93%):

The dashboard displays the retention value at exactly the midpoint of the video ($x = 0.5$). $e^{(-0.8)} = 0.44933 \rightarrow 44.93\%$. This value shown in the Video Deep Insights panel is **calculated**, not fetched from Facebook — it is an approximation of typical video ad retention. Real per-quartile data (p25/p50/p75/p100) would require Facebook's video_p25_watched_actions fields which require additional API permissions.

★ Quick Reference — All Formulas

Metric	Formula	Data Source
KPI Impressions (1H)	SUM(impressions) in [maxDate–1h, maxDate]	ad_metrics
KPI Views (1H)	SUM(video_views) if >0, else SUM(impressions)	ad_metrics
KPI Engagement Rate	$\text{SUM(likes+comments+shares+saves)} \div \text{SUM(impressions)} \times 100\%$	ad_metrics
KPI CTR	$\text{SUM(clicks)} \div \text{SUM(impressions)} \times 100\%$	ad_metrics
KPI Followers Gained	SUM(followers_gained)	ad_metrics
KPI Revenue (1H)	SUM(conversion_value)	ad_metrics
KPI Delta Arrow	$(\text{current} - \text{previous}) \div \text{previous} \times 100\%$	computed
TREND Views chart	SUM(video_views >0 ? video_views : impressions) per BST hour bucket	ad_metrics
TREND Engagement chart	SUM(likes+comments+shares+saves) per BST hour bucket	ad_metrics
TREND Watch time chart	SUM(video_views × avg_watch_seconds) per hour (total seconds)	ad_metrics
POST Views (1H)	SUM(video_views >0 ? video_views : impressions) in last window, per ad	ad_metrics (ad_id ≠ null)
POST ER	$\text{SUM(eng)} \div \text{SUM(impressions)} \times 100\%$, per ad in last window	ad_metrics
POST Velocity %	$(\text{lastViews} - \text{prevViews}) \div \text{prevViews} \times 100\%$	computed
POST Virality	$\text{SUM(shares)} \div \text{SUM(impressions)} \times 100\%$, per ad in last window	ad_metrics
VIRAL Lift score	$(\text{recent}/2 - \text{baseline}) \div \text{baseline}$; baseline = 6h avg/hr	computed
VIRAL Spike alert	lift > 2.0 (recent rate is 3× the baseline rate)	computed
HEAT Heatmap cell	AVG(impressions >0 ? impressions : engagement) per (platform, BST hour)	ad_metrics

VIDEO	Completion rate	$\text{video_completions} \div \text{video_views} \times 100\%$	ad_metrics
VIDEO	50% Retention	$e^{(-0.5 \times 1.6)} \times 100 = 44.93\%$ (exponential decay model)	model (approx.)